IM-102 Advanced Database Systems

Capstone Project Documentation

Student Name, Student ID:

Agapito, Zyron G. (24-3332)

Angeles, Jefferson T. (24-3494)

Callos, Joseph S. (24-3250)

Dela Cruz, Fielbert D. (24-3483)

Lara, Jian Lee D. (24-3397)

Terrago, Louis Miguel M. (24-3107)

Vecina, John Francis V. (24-3230)

Project Title: IT Helpdesk Ticketing System: Logs user issues, assigns IT technicians, tracks resolution SLAs, and manages hardware asset inventory.

Submission Date: March 20, 2026

Instructor: Mr. Rommel Apostol

Executive Summary

This capstone project presents the design, development, and optimization of an IT Helpdesk Ticketing System built on MySQL 8.0. The system addresses a common organizational challenge: IT support requests arriving through scattered channels, causing delays, missed issues, and uneven technician workloads. The database centralizes all IT support operations by allowing employees to submit tickets linked directly to their assigned hardware assets. Upon submission, the system automatically sets a 48-hour SLA deadline, assigns the least-loaded available technician, and logs every action for full traceability. Priority escalation runs hourly through scheduled events, and resolved tickets are archived automatically after seven days to maintain database performance.

Key features implemented include seven normalized tables — users, technicians, assets, tickets, assignments, ticket_logs, and archived_tickets — four stored procedures including a smart ticket creation mechanism that accepts email and asset code instead of raw IDs, five views for role-specific data access including email masking for privacy compliance under RA 10173, and five triggers for automation and validation including prevention of edits on closed tickets. Performance was improved by adding composite and targeted indexes, reducing query execution time by 99.4% on the departmental activity report. Security is enforced through six MySQL user roles ranging from full database administrator access down to read-only analyst access, with connection-level SSL enforcement for application users.

DELIVERABLE 1: Schema Design Document

1.1 System Overview

Domain Description:

This capstone project is to make an IT Helpdesk Ticketing System essential for managing IT support in any organization. Offices and schools often deal with tech issues coming in from all directions: emails, calls, even handwritten notes. Things slip through the cracks. With a centralized system, every problem lands in one place, so nothing gets lost or forgotten. The database lets you assign tickets to technicians, keep an eye on each issue's status, and track hardware assets all in one spot. The helpdesk runs smoother, issues get resolved faster, and chaos turns into order.

Key Stakeholders:

1. Primary users — Employees and office staff who submit IT support tickets when they encounter hardware or software problems.
2. IT Technicians — Specialized support staff assigned to diagnose and resolve reported issues.
3. IT Managers / Administrators — Supervisors who monitor ticket queues, technician workloads, and SLA compliance.
4. System Analysts — Read-only users responsible for generating performance and incident reports.

Business Requirements:

1. The system must allow employees to report IT issues and link their concerns directly to specific company hardware or devices.
2. The system must automatically assign new tickets to the technician with the fewest pending tasks to ensure the workload is balanced.
3. The system must automatically set a 48-hour deadline for every new ticket to make sure issues are handled on time.
4. The system must automatically increase the priority level of a ticket (for example, from High to Critical) if the technician fails to resolve it before the deadline.
5. The system must record a complete history of every action, status change, and priority adjustment made on a ticket.
6. The system must automatically move resolved tickets to an archive table after seven days to keep the main database fast and uncluttered.
7. The system must lock tickets once they are permanently closed so that past records cannot be edited or tampered with.
8. The system must mask sensitive employee information, such as full email addresses, to protect user privacy.
9. The system must enforce strict user roles so that IT staff, managers, and administrators can only access and modify the data allowed for their specific job.
10. The system must balance the IT workload by automatically assigning new requests to the technician who currently has the fewest open tickets.

1.2 Entity-Relationship Diagram

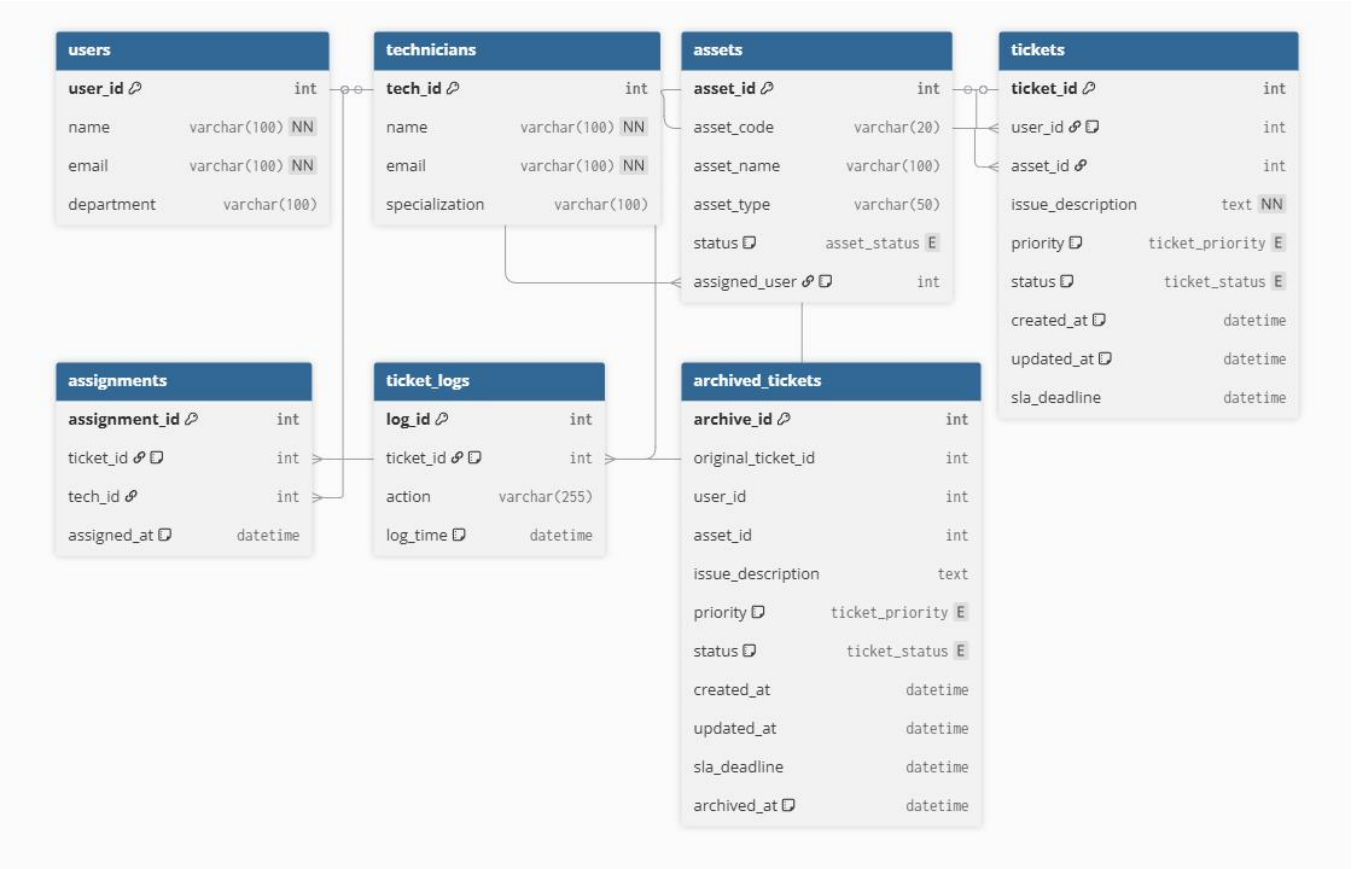

Figure 1: Complete ER Diagram showing all entities, attributes, and relationships

ER Diagram Description:

The ER diagram for the IT Helpdesk Ticketing System lays out all the key pieces of information the system needs and shows how everything connects. Basically, the database helps organize IT issues that employees report, lets you assign technicians to handle them, track how things progress, and keeps records of solved tickets for later. You'll see several main groups in the diagram: users, technicians, assets, tickets, assignments, ticket_logs, and archived_tickets.

The users section holds details about employees who report problems. Technicians are where info about IT staff lives; these are the people fixing things. Assets keep track of hardware, like

computers or laptops tied to a ticket. Tickets are really the heart of it all; that's where every helpdesk request lands, with details about what went wrong, its current status, how urgent it is, and when it happened.

The ER diagram basically maps out how everything fits together. Users can make as many tickets as they want, but each ticket belongs to just one user. It's a one-to-many setup. Users also get assigned different assets, but each asset only goes to one user at a time. Assets aren't static; they can link to multiple tickets because the same device might have problems pop up every now and then.

The assignments table steps in to show which technician is handling which ticket. One technician can juggle several assignments, but each assignment ties back to just one ticket and one technician. Then there's the ticket_logs table, which keeps track of updates or actions — basically, every little change made on a ticket. A ticket can rack up multiple logs, and technicians can create a bunch of log entries themselves. Finally, the archived_tickets table is all about keeping things tidy. Instead of deleting old, resolved tickets, the system stores them here. It keeps the database neat, but you still get to hang onto all the important records.

We kept the design straightforward so anyone can grasp it and explain it without hassle. Still, it covers all the key functions the system needs. The tickets table sits right at the heart of the database because, honestly, that's where everything starts in the helpdesk process. We added separate tables assignments and ticket_logs to make it obvious who's working on what and to track how tickets move along. There's also an archived_tickets table for keeping past records and tracking history. In the end, this ER diagram lays out a clear, organized way to store and handle IT support requests so things run smoothly.

1.3 Normalization Analysis

Normalization Level Achieved: Third Normal Form (3NF)

Normal Form	Status	Justification
1NF	YES	The data is organized so that each field contains only one value, and there are no repeating or grouped columns. All tables use atomic values and have a defined primary key.
2NF	YES	All columns in each table depend entirely on the primary key, meaning there are no partial dependencies. Every non-key column is fully dependent on the whole primary key.
3NF	YES	The tables are structured so that no column depends on another non-key column, ensuring there are no transitive dependencies. All non-key attributes depend solely on the primary key.

Table 1: Normalization compliance checklist

Denormalization Decisions (if any):

No denormalization was applied in the system. The database maintains a fully normalized structure up to Third Normal Form (3NF) to ensure data consistency, reduce redundancy, and maintain integrity. Performance optimization is handled through indexing and query optimization instead of denormalization.

No denormalization techniques were applied in the database design. The system strictly follows Third Normal Form (3NF) to ensure data integrity, eliminate redundancy, and maintain consistency across all tables.

Denormalization was not necessary because the current system workload does not require heavy read optimization that would justify redundancy. However, in future large-scale deployments, denormalization may be considered for reporting tables or dashboards to improve query performance.

1.4 Table Definitions

Table 1: users

Purpose: Stores employees who report IT issues in the system.

Column	Data Type	Constraints	Default	Description
user_id	INT	PK, AUTO_INCREMENT	—	Unique user identifier
name	VARCHAR(100)	NOT NULL	—	Full name of the user
email	VARCHAR(100)	NOT NULL, UNIQUE	—	User email address
department	VARCHAR(100)	—	NULL	Department of the user

Indexes:

- PRIMARY KEY (user_id)
- UNIQUE INDEX (email)

Foreign Key Relationships:

- None

Table 2: technicians
Purpose: Stores IT technicians responsible for resolving tickets.

Column	Data Type	Constraints	Default	Description
tech_id	INT	PK, AUTO_INCREMENT	—	Unique technician ID
name	VARCHAR(100)	NOT NULL	—	Technician name
email	VARCHAR(100)	NOT NULL, UNIQUE	—	Technician email
specialization	VARCHAR(100)	—	NULL	Area of expertise (Hardware, Software, Networking)

Indexes:

- PRIMARY KEY (tech_id)
- UNIQUE INDEX (email)

Foreign Key Relationships:

- None

Table 3: assets

Purpose: Stores hardware assets such as computers, printers, and laptops.

Column	Data Type	Constraints	Default	Description
asset_id	INT	PK, AUTO_INCREMENT	—	Unique asset ID
asset_code	VARCHAR(20)	UNIQUE	—	Asset identifier (e.g., CL-101)
asset_name	VARCHAR(100)	—	NULL	Name of asset
asset_type	VARCHAR(50)	—	NULL	Type of asset
status	ENUM	—	'In Use'	Asset status: Available, In Use, Under Repair
assigned_user	INT	FK	NULL	Linked user

Indexes:

- PRIMARY KEY (asset_id)
- UNIQUE INDEX (asset_code)

Foreign Key Relationships:

- FK: assigned_user REFERENCES users(user_id)

Table 4: tickets

Purpose: Core table storing all IT support issues.

Column	Data Type	Constraints	Default	Description
ticket_id	INT	PK, AUTO_INCREMENT	—	Unique ticket ID
user_id	INT	FK	NULL	User who reported
asset_id	INT	FK	NULL	Related asset
issue_description	TEXT	NOT NULL	—	Description of issue
priority	ENUM	—	'Low'	Ticket priority: Low, Medium, High, Critical
status	ENUM	—	'Open'	Ticket status: Open, In Progress, Resolved, Closed
created_at	DATETIME	—	CURRENT_TIMESTAMP	Creation timestamp
updated_at	DATETIME	—	CURRENT_TIMESTAMP ON UPDATE	Last update
sla_deadline	DATETIME	—	NULL	SLA deadline (auto-set by trigger)

Indexes:

- PRIMARY KEY (ticket_id)
- INDEX idx_ticket_status_priority (status, priority)

Foreign Key Relationships:

- FK: user_id REFERENCES users(user_id) ON DELETE CASCADE
- FK: asset_id REFERENCES assets(asset_id)

Table 5: assignments

Purpose: Links tickets to technicians.

Column	Data Type	Constraints	Default	Description
assignment_id	INT	PK, AUTO_INCREMENT	—	Unique assignment ID
ticket_id	INT	FK	NULL	Linked ticket
tech_id	INT	FK	NULL	Assigned technician
assigned_at	DATETIME	—	CURRENT_TIMESTAMP	Assignment time

Indexes:

- PRIMARY KEY (assignment_id)

Foreign Key Relationships:

- FK: ticket_id REFERENCES tickets(ticket_id) ON DELETE CASCADE
- FK: tech_id REFERENCES technicians(tech_id)

Table 6: ticket_logs

Purpose: Stores activity history for each ticket.

Column	Data Type	Constraints	Default	Description
log_id	INT	PK, AUTO_INCREMENT	—	Unique log ID
ticket_id	INT	FK	NULL	Related ticket
action	VARCHAR(255)	—	NULL	Action description
log_time	DATETIME	—	CURRENT_TIMESTAMP	Log timestamp

Indexes:

- PRIMARY KEY (log_id)

Foreign Key Relationships:

- FK: ticket_id REFERENCES tickets(ticket_id) ON DELETE CASCADE

Table 7: archived_tickets

Purpose: Stores resolved tickets moved after 7 days.

Column	Data Type	Constraints	Default	Description
archive_id	INT	PK, AUTO_INCREMENT	—	Archive ID
original_ticket_id	INT	—	NULL	Original ticket ID
user_id	INT	—	NULL	Reporting user
asset_id	INT	—	NULL	Related asset
issue_description	TEXT	—	NULL	Issue details
priority	ENUM	—	NULL	Priority
status	ENUM	—	NULL	Status
created_at	DATETIME	—	NULL	Created time
updated_at	DATETIME	—	NULL	Updated time
sla_deadline	DATETIME	—	NULL	SLA deadline
archived_at	DATETIME	—	CURRENT_TIMESTAMP	Archive time

Indexes:

- PRIMARY KEY (archive_id)

Foreign Key Relationships:

- None

1.5 Constraints Summary

Constraint Type	Table.Column	Purpose
PRIMARY KEY	users.user_id	Unique identifier for each user
PRIMARY KEY	technicians.tech_id	Unique identifier for each technician
PRIMARY KEY	assets.asset_id	Unique identifier for each asset
PRIMARY KEY	tickets.ticket_id	Unique identifier for each ticket
PRIMARY KEY	assignments.assignment_id	Unique identifier for each assignment
PRIMARY KEY	ticket_logs.log_id	Unique identifier for each log
PRIMARY KEY	archived_tickets.archive_id	Unique identifier for archived records
FOREIGN KEY	assets.assigned_user	Links asset to a user
FOREIGN KEY	tickets.user_id	Links ticket to reporting user
FOREIGN KEY	tickets.asset_id	Links ticket to asset
FOREIGN KEY	assignments.ticket_id	Links assignment to ticket
FOREIGN KEY	assignments.tech_id	Links assignment to technician
FOREIGN KEY	ticket_logs.ticket_id	Links logs to ticket
NOT NULL	users.name	Ensures user has a name
NOT NULL	users.email	Ensures user email is required
NOT NULL	technicians.name	Ensures technician name is required
NOT NULL	technicians.email	Ensures technician email is required
NOT NULL	tickets.issue_description	Ensures issue details are provided
UNIQUE	users.email	Prevent duplicate user emails
UNIQUE	technicians.email	Prevent duplicate technician emails
UNIQUE	assets.asset_code	Ensures each asset has a unique identifier
CHECK (ENUM)	assets.status	Restricts values to Available, In Use, Under Repair
CHECK (ENUM)	tickets.priority	Restricts values to Low, Medium, High, Critical
CHECK (ENUM)	tickets.status	Restricts values to Open, In Progress, Resolved, Closed

Table 3: Complete constraints inventory

1.6 Scalability Planning

Current Database Size Estimates:

Expected number of records per table:

- users: 10 – 20 records
- technicians: 5 – 10 records
- assets: 10 – 20 records
- tickets: 10 – 20 records
- assignments: 10 – 20 records
- ticket_logs: 20 – 50 records
- archived_tickets: 10 – 20 records

Estimated storage requirements:

- Current dataset size: less than 10MB
- Due to small sample size, storage usage is minimal and suitable for testing and demonstration purposes

Growth projections:

- Although the current dataset is small, the system is designed to scale
- In real-world usage, ticket and log data will grow rapidly over time
- Expected growth will mainly come from:
  - Continuous ticket creation
  - Logging of actions per ticket
  - Accumulation of archived records

Scalability Strategies:

Archiving plan for historical data:

- Implement automated archiving using scheduled events
- Tickets marked as Resolved are moved to archived_tickets after 7 days
- This prevents the tickets table from becoming too large
- Maintains system performance while preserving historical records

Indexing strategy for large tables:

- Index implemented: idx_ticket_status_priority (status, priority) for faster filtering
- Primary keys ensure fast data retrieval
- Foreign keys support efficient JOIN operations
- Future improvements:

- Index on sla_deadline for SLA monitoring
- Index on created_at for time-based queries

Query optimization considerations:

- Use indexed columns in WHERE and JOIN conditions
- Avoid unnecessary full table scans
- Use views for simplified and reusable queries
- Limit returned data using filters instead of retrieving all records
- Automate heavy operations (archiving, SLA escalation) using events to reduce runtime load

DELIVERABLE 2: SQL Scripts

2.1 DDL Scripts (CREATE TABLE Statements)

File: *01_create_tables.sql*

```
CREATE DATABASE IF NOT EXISTS it_helpdesk;
USE it_helpdesk;

CREATE TABLE users (
   user_id    INT AUTO_INCREMENT PRIMARY KEY,
   name       VARCHAR(100) NOT NULL,
   email      VARCHAR(100) NOT NULL UNIQUE,
   department VARCHAR(100)
);

CREATE TABLE technicians (
   tech_id        INT AUTO_INCREMENT PRIMARY KEY,
   name           VARCHAR(100) NOT NULL,
   email          VARCHAR(100) NOT NULL UNIQUE,
   specialization VARCHAR(100)
);

CREATE TABLE assets (
   asset_id      INT AUTO_INCREMENT PRIMARY KEY,
   asset_code    VARCHAR(20) UNIQUE,
   asset_name    VARCHAR(100),
   asset_type    VARCHAR(50),
   status        ENUM('Available','In Use','Under Repair') DEFAULT 'In Use',
   assigned_user INT,
   FOREIGN KEY (assigned_user) REFERENCES users(user_id)
);

CREATE TABLE tickets (
   ticket_id        INT AUTO_INCREMENT PRIMARY KEY,
   user_id          INT,
   asset_id         INT,
   issue_description TEXT NOT NULL,
   priority         ENUM('Low','Medium','High','Critical') DEFAULT 'Low',
   status           ENUM('Open','In Progress','Resolved','Closed') DEFAULT 'Open',
   created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
   updated_at       DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
```

```
    sla_deadline      DATETIME,
    FOREIGN KEY (user_id)  REFERENCES users(user_id)  ON DELETE CASCADE,
    FOREIGN KEY (asset_id) REFERENCES assets(asset_id)
);

CREATE TABLE archived_tickets (
    archive_id        INT AUTO_INCREMENT PRIMARY KEY,
    original_ticket_id INT,
    user_id INT, asset_id INT,
    issue_description TEXT,
    priority ENUM('Low','Medium','High','Critical'),
    status   ENUM('Open','In Progress','Resolved','Closed'),
    created_at DATETIME, updated_at DATETIME, sla_deadline DATETIME,
    archived_at DATETIME DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE assignments (
    assignment_id INT AUTO_INCREMENT PRIMARY KEY,
    ticket_id INT, tech_id INT,
    assigned_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ticket_id) REFERENCES tickets(ticket_id) ON DELETE
CASCADE,
    FOREIGN KEY (tech_id)   REFERENCES technicians(tech_id)
);

CREATE TABLE ticket_logs (
    log_id    INT AUTO_INCREMENT PRIMARY KEY,
    ticket_id INT,
    action    VARCHAR(255),
    log_time  DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ticket_id) REFERENCES tickets(ticket_id) ON DELETE
CASCADE
);

CREATE INDEX idx_ticket_status_priority ON tickets(status, priority);
```

2.2 DML Scripts (INSERT Sample Data)

File: *02_insert_sample_data.sql*

```
USE it_helpdesk;

-- USERS (12 records)
INSERT INTO users (name, email, department) VALUES
('Zyron Agapito',      'zyron@email.com',       'IT'),
('Jefferson Angeles',  'jefferson@email.com',   'Finance'),
('Joseph Callos',      'joseph@email.com',      'HR'),
('Fielbert Dela Cruz', 'fielbert@email.com',    'Operations'),
('Jian Lee Lara',      'jianlee@email.com',     'Marketing'),
('Louis Terrago',      'louis@email.com',       'Accounting'),
('John Francis Vecina','johnfrancis@email.com', 'IT'),
('Clara Santos',       'clara@email.com',       'Admin'),
('Mark Reyes',         'mark@email.com',        'Finance'),
('Nina Cruz',          'nina@email.com',        'Operations'),
('Paolo Bautista',     'paolo@email.com',       'HR'),
('Sophia Ramos',       'sophia@email.com',      'Marketing');

-- TECHNICIANS (5 records)
INSERT INTO technicians (name, email, specialization) VALUES
('Carlos Magtibay', 'carlos@itstaff.com', 'Hardware'),
('Diana Poblete',   'diana@itstaff.com',  'Software'),
('Edwin Castillo',  'edwin@itstaff.com',  'Networking'),
('Fiona Navarro',   'fiona@itstaff.com',  'Hardware'),
('George Salazar',  'george@itstaff.com', 'Software');

-- ASSETS (14 records)
INSERT INTO assets (asset_code, asset_name, asset_type, status, assigned_user)
VALUES
('CL-101','PC 1','PC','In Use',1), ('CL-102','PC 2','PC','In Use',2),
('CL-103','PC 3','PC','In Use',3), ('CL-104','PC 4','PC','Under Repair',4),
('CL-105','PC 5','PC','In Use',5), ('CL-106','PC 6','PC','In Use',6),
('CL-107','PC 7','PC','In Use',7), ('CL-108','PC 8','PC','In Use',8),
('CL-109','PC 9','PC','Under Repair',9), ('CL-110','PC 10','PC','In Use',10),
('CL-111','PC 11','PC','Available',NULL), ('CL-112','PC 12','PC','In Use',11),
('CL-113','Printer A','Printer','In Use',12), ('CL-114','Laptop 1','Laptop','In Use',1);

-- TICKETS (16 records)
INSERT INTO tickets (user_id, asset_id, issue_description, priority, status) VALUES
(1,1,'PC not turning on','High','Open'),
(2,2,'Blue screen of death on startup','Critical','In Progress'),
(3,3,'Cannot connect to the internet','Medium','Open'),
(4,4,'Keyboard not responding','Low','Resolved'),
```

```
(5,5,'Monitor flickering on and off','Medium','Open'),
(6,6,'Slow performance and high CPU usage','High','In Progress'),
(7,7,'Printer not detected by the system','Low','Open'),
(8,8,'Application crashes immediately on launch','High','Resolved'),
(9,9,'Cannot log in to domain account','Critical','Open'),
(3,3,'No sound output from speakers','Low','Resolved'),
(1,14,'Laptop battery not charging','High','In Progress'),
(10,10,'USB ports not working on any device','Medium','Open'),
(2,2,'Outlook not syncing emails','Medium','In Progress'),
(5,5,'Hard disk making loud clicking noise','Critical','Open'),
(11,12,'Display driver crashes on video playback','High','Open'),
(4,13,'Printer jams on every print job','Low','Open');

-- Minimum 10-20 records per table. See 02_insert_sample_data.sql for
-- full archived_tickets (10 records), assignments (16), and ticket_logs (20).
```

2.3 Complex Queries

File: *03_complex_queries.sql*

Query 1: Multi-Table JOIN with Aggregation

Purpose: Show total tickets per department to identify which departments generate the most IT issues.

```
SELECT
   u.department,
   COUNT(t.ticket_id)                          AS total_tickets,
   SUM(t.priority = 'Critical')                AS critical_count,
   SUM(t.status = 'Resolved')                  AS resolved_count,
   ROUND(SUM(t.status='Resolved')/NULLIF(COUNT(t.ticket_id),0)*100,2)
                                               AS resolution_percentage
FROM users u
LEFT JOIN tickets t ON u.user_id = t.user_id
GROUP BY u.department
ORDER BY total_tickets DESC;
```

Expected Output Sample:

department	total_tickets	critical_count	resolved_count	resolution_percentage
IT	3	0	0	0.00
Finance	3	2	0	0.00
HR	3	0	1	33.33
Operations	3	0	1	33.33
Marketing	2	1	0	0.00
Accounting	1	0	0	0.00
Admin	1	0	1	100.00

Table 4: Sample query results

Query 2: Technician Workload Report

Purpose: Show how many tickets each technician is handling to identify workload distribution.

```
SELECT tech.name AS technician_name, tech.specialization,
   COUNT(a.assignment_id) AS total_assigned,
   SUM(t.status = 'Open') AS open_tickets,
   SUM(t.status = 'In Progress') AS in_progress_tickets,
   SUM(t.status = 'Resolved') AS resolved_tickets
FROM technicians tech
LEFT JOIN assignments a ON tech.tech_id = a.tech_id
LEFT JOIN tickets t ON a.ticket_id = t.ticket_id
GROUP BY tech.tech_id, tech.name, tech.specialization
ORDER BY total_assigned DESC;
```

Query 3: Window Function — Ticket Ranking per Department

Purpose: Rank tickets within each department based on priority and creation time for better prioritization.

```
SELECT u.department, t.ticket_id, u.name AS reported_by,
   t.issue_description, t.priority, t.status,
   ROW_NUMBER() OVER (
      PARTITION BY u.department
      ORDER BY FIELD(t.priority,'Critical','High','Medium','Low'), t.created_at ASC
```

```
    ) AS priority_rank
FROM tickets t
JOIN users u ON t.user_id = u.user_id
WHERE t.status NOT IN ('Resolved','Closed')
ORDER BY u.department, priority_rank;
```

Query 4: CTE — Asset Trouble Report

Purpose: Identify assets with multiple reported issues, indicating possible faulty hardware.

```
WITH asset_ticket_counts AS (
    SELECT a.asset_id, a.asset_code, a.asset_name, a.asset_type,
           a.status AS asset_status, COUNT(t.ticket_id) AS ticket_count,
           MAX(t.created_at) AS last_reported
    FROM assets a LEFT JOIN tickets t ON a.asset_id = t.asset_id
    GROUP BY a.asset_id, a.asset_code, a.asset_name, a.asset_type, a.status
)
SELECT * FROM asset_ticket_counts
WHERE ticket_count > 1
ORDER BY ticket_count DESC, last_reported DESC;
```

Query 5: Subquery with EXISTS — Active Users with Open Tickets

Purpose: List users who currently have unresolved tickets.

```
SELECT u.user_id, u.name, u.email, u.department
FROM users u
WHERE EXISTS (
    SELECT 1 FROM tickets t
    WHERE t.user_id = u.user_id
      AND t.status NOT IN ('Resolved','Closed')
)
ORDER BY u.department, u.name;
```

Query 6: UNION — Combined Ticket Activity Feed

Purpose: Display both active and archived tickets in a single timeline for complete monitoring.

```
SELECT 'Active' AS source, ticket_id AS id, user_id,
      issue_description, priority, status, created_at FROM tickets
UNION ALL
SELECT 'Archived' AS source, original_ticket_id AS id, user_id,
      issue_description, priority, status, created_at FROM archived_tickets
ORDER BY created_at DESC;
```

Query 7: GROUP BY + HAVING — High-Volume Reporters

Purpose: Identify users who frequently report issues, possibly indicating recurring problems.

```
SELECT u.user_id, u.name, u.department,
   COUNT(t.ticket_id) AS total_tickets_filed,
   SUM(t.priority IN ('High','Critical')) AS high_priority_count,
   MAX(t.created_at) AS most_recent_ticket
FROM users u JOIN tickets t ON u.user_id = t.user_id
GROUP BY u.user_id, u.name, u.department
HAVING COUNT(t.ticket_id) > 1
ORDER BY total_tickets_filed DESC;
```

Query 8: SLA Status Summary per Priority

Purpose: Analyze how many tickets per priority level are overdue vs. within SLA to evaluate system performance.

```
SELECT priority, COUNT(*) AS total_tickets,
   SUM(IF(NOW()>sla_deadline AND status NOT IN ('Resolved','Closed'),1,0)) AS
overdue_tickets,
   SUM(IF(NOW()<=sla_deadline OR status IN ('Resolved','Closed'),1,0)) AS within_sla,
   ROUND(SUM(IF(NOW()>sla_deadline AND status NOT IN
('Resolved','Closed'),1,0))
   /COUNT(*)*100, 2) AS overdue_percentage
FROM tickets
GROUP BY priority ORDER BY overdue_percentage DESC;
```

2.4 Stored Procedures

File: *04_stored_procedures.sql*

Function: get_least_loaded_tech()

Purpose: Returns the tech_id of the technician with the fewest current assignments. Used internally by the auto_assign_technician trigger.

```
DELIMITER $$
CREATE FUNCTION get_least_loaded_tech()
RETURNS INT DETERMINISTIC
BEGIN
   DECLARE selected_tech INT;
   SELECT t.tech_id INTO selected_tech
   FROM technicians t LEFT JOIN assignments a ON t.tech_id = a.tech_id
   GROUP BY t.tech_id ORDER BY COUNT(a.assignment_id) ASC LIMIT 1;
   RETURN selected_tech;
END$$
DELIMITER ;
```

Procedure 1: create_ticket_smart

Purpose: Allows employees to create a ticket using their email address and asset code instead of raw IDs. Internally resolves user_id and asset_id before inserting. Triggers handle SLA and assignment automatically.

```
DELIMITER $$
CREATE PROCEDURE create_ticket_smart(
   IN p_email VARCHAR(100), IN p_asset_code VARCHAR(20), IN p_issue TEXT
)
BEGIN
   DECLARE v_user_id INT; DECLARE v_asset_id INT;
   SELECT user_id  INTO v_user_id  FROM users  WHERE email=p_email      LIMIT 1;
   SELECT asset_id INTO v_asset_id FROM assets WHERE asset_code=p_asset_code
LIMIT 1;
   INSERT INTO tickets (user_id, asset_id, issue_description)
```

```
    VALUES (v_user_id, v_asset_id, p_issue);
    INSERT INTO ticket_logs (ticket_id, action)
    VALUES (LAST_INSERT_ID(), 'Ticket created via smart procedure');
END$$
DELIMITER ;
```

Test Case: CALL create_ticket_smart('zyron@email.com', 'CL-101', 'PC not turning on');

Procedure 2: resolve_ticket

Purpose: Marks a ticket as Resolved and logs the resolution action.

```
DELIMITER $$
CREATE PROCEDURE resolve_ticket(IN p_ticket_id INT)
BEGIN
    UPDATE tickets SET status='Resolved' WHERE ticket_id=p_ticket_id;
    INSERT INTO ticket_logs (ticket_id, action) VALUES (p_ticket_id,'Ticket resolved');
END$$
DELIMITER ;
```

Test Case: CALL resolve_ticket(1);

Procedure 3: auto_escalate_sla

Purpose: Escalates priority of all open/in-progress tickets whose SLA deadline has passed.

Escalation: Low → Medium → High → Critical. Called hourly by the sla_auto_escalation event.

```
DELIMITER $$
CREATE PROCEDURE auto_escalate_sla()
BEGIN
    UPDATE tickets SET priority=CASE
        WHEN priority='Low' THEN 'Medium' WHEN priority='Medium' THEN 'High'
        WHEN priority='High' THEN 'Critical' ELSE priority END
    WHERE status NOT IN ('Resolved','Closed') AND NOW() > sla_deadline;
END$$
DELIMITER ;
```

Test Case: CALL auto_escalate_sla();

Procedure 4: archive_old_tickets_proc

Purpose: Moves resolved tickets older than 7 days into archived_tickets, then deletes them from the main table. Called daily by the archive_old_tickets event.

```
DELIMITER $$
CREATE PROCEDURE archive_old_tickets_proc()
BEGIN
   INSERT INTO archived_tickets(
      original_ticket_id, user_id, asset_id, issue_description,
      priority, status, created_at, updated_at, sla_deadline)
   SELECT ticket_id, user_id, asset_id, issue_description,
         priority, status, created_at, updated_at, sla_deadline
   FROM tickets WHERE status='Resolved' AND updated_at <= NOW()-INTERVAL 7
DAY;
   DELETE FROM tickets WHERE status='Resolved' AND updated_at <= NOW()-
INTERVAL 7 DAY;
END$$
DELIMITER ;
```

Test Case: CALL archive_old_tickets_proc();

2.5 Views

File: *05_views.sql*

The following five views are created for the IT Helpdesk Ticketing System. Each view serves a specific reporting or security purpose — simplifying complex JOIN queries for end users and restricting direct access to sensitive base tables based on the principle of least privilege.

View 1: active_tickets

Purpose: Provides a simplified, joined view of all currently active tickets by combining the tickets, users, and assets tables.

```
CREATE VIEW active_tickets AS
SELECT t.ticket_id, u.name AS reported_by, u.department, a.asset_code,
    a.asset_name, t.issue_description, t.priority, t.status,
    t.created_at, t.sla_deadline
FROM tickets t
JOIN users  u ON t.user_id  = u.user_id
JOIN assets a ON t.asset_id = a.asset_id;
```

Security Benefit: Abstracts the underlying table structure from end users. Technicians and managers can query ticket data without having direct SELECT access to the base tables — reducing the risk of accidental data exposure or manipulation.

Test Case: SELECT * FROM active_tickets; / SELECT * FROM active_tickets WHERE status = 'Open';

ticket_id	reported_by	department	asset_code	priority	status
1	Zyron Agapito	IT	CL-101	High	Open
2	Jefferson Angeles	Finance	CL-102	Critical	In Progress
9	Nina Cruz	Operations	CL-109	Critical	Open

View 2: priority_tickets

Purpose: Surfaces only Critical priority tickets that are still Open, ordered by SLA deadline ascending.

```
CREATE VIEW priority_tickets AS
SELECT t.ticket_id, u.name AS reported_by, u.department, a.asset_code,
    t.issue_description, t.priority, t.sla_deadline,
    TIMESTAMPDIFF(HOUR, NOW(), t.sla_deadline) AS hours_until_breach
FROM tickets t
JOIN users u ON t.user_id=u.user_id JOIN assets a ON t.asset_id=a.asset_id
WHERE t.priority='Critical' AND t.status='Open'
ORDER BY t.sla_deadline ASC;
```

Security Benefit: Restricts visibility to only the most actionable subset of data. Roles cannot see lower-priority tickets, archived data, or unnecessary user contact details.

Test Case: SELECT * FROM priority_tickets;

ticket_id	reported_by	asset_code	priority	sla_deadline	hours_until_breach
9	Nina Cruz	CL-109	Critical	2026-03-03 12:00:00	8
14	Jian Lee Lara	CL-105	Critical	2026-03-04 10:00:00	30

View 3: archived_view

Purpose: Provides read-only access to the complete history of resolved and archived tickets, joined with user and asset information.

```
CREATE VIEW archived_view AS
SELECT ar.archive_id, ar.original_ticket_id, u.name AS reported_by,
    u.department, a.asset_code, ar.issue_description,
    ar.priority, ar.status, ar.created_at, ar.sla_deadline, ar.archived_at
FROM archived_tickets ar
LEFT JOIN users  u ON ar.user_id  = u.user_id
LEFT JOIN assets a ON ar.asset_id = a.asset_id;
```

Security Benefit: Separates historical data access from live operational data. Analysts can query history without any path to modify live tickets.

Test Case: SELECT * FROM archived_view ORDER BY archived_at DESC;

archive_id	original_ticket_id	reported_by	asset_code	priority	archived_at
10	60	Sophia Ramos	CL-113	Medium	2026-02-04 08:00:00
9	59	Paolo Bautista	CL-112	Critical	2026-02-01 08:00:00

View 4: technician_dashboard

Purpose: Aggregates each technician's current assignment counts broken down by ticket status.

```
CREATE VIEW technician_dashboard AS
SELECT tech.tech_id, tech.name AS technician_name, tech.specialization,
    COUNT(a.assignment_id) AS total_assigned,
    SUM(t.status='Open') AS open_tickets,
    SUM(t.status='In Progress') AS in_progress_tickets,
    SUM(t.status='Resolved') AS resolved_tickets
FROM technicians tech
LEFT JOIN assignments a ON tech.tech_id = a.tech_id
LEFT JOIN tickets     t ON a.ticket_id  = t.ticket_id
GROUP BY tech.tech_id, tech.name, tech.specialization;
```

Security Benefit: Exposes only aggregate workload counts — not individual ticket details, user names, or issue descriptions from other departments.

Test Case: SELECT * FROM technician_dashboard ORDER BY open_tickets DESC;

tech_id	technician_name	specialization	total_assigned	open_tickets	in_progress_tickets
1	Carlos Magtibay	Hardware	4	2	1
2	Diana Poblete	Software	3	1	1
3	Edwin Castillo	Networking	3	1	1

View 5: vw_users_public

Purpose: Exposes a masked version of the users table, replacing the full email with a partially hidden format (e.g., jos***@email.com).

```
CREATE VIEW vw_users_public AS
SELECT user_id, name,
   CONCAT(LEFT(email,3),'***@',SUBSTRING_INDEX(email,'@',-1)) AS
email_masked,
   department
FROM users;

GRANT SELECT ON it_helpdesk.vw_users_public TO 'technician'@'localhost';
```

Security Benefit: Prevents email harvesting. Implements data minimization in line with the Data Privacy Act (RA 10173). Technicians can identify who filed a ticket without seeing full emails.

Test Case: SELECT * FROM vw_users_public;

user_id	name	email_masked	department
1	Zyron Agapito	zyr***@email.com	IT
2	Jefferson Angeles	jef***@email.com	Finance
3	Joseph Callos	jos***@email.com	HR

2.6 Triggers

File: *06_triggers.sql*

The following five triggers automate key system behaviors, enforce data validation rules, and maintain a complete audit trail for every ticket lifecycle event. They are designed to fire without any manual intervention — ensuring consistent system behavior regardless of how data is inserted or updated.

Trigger 1: set_sla_deadline

Event: BEFORE INSERT on tickets

Purpose: Automatically sets the SLA deadline to exactly 48 hours from the moment a ticket is created.

```
DELIMITER $$
CREATE TRIGGER set_sla_deadline
BEFORE INSERT ON tickets FOR EACH ROW
BEGIN
   SET NEW.sla_deadline = DATE_ADD(NOW(), INTERVAL 48 HOUR);
END$$
DELIMITER ;
```

Test Case:

```
CALL create_ticket_smart('zyron@email.com','CL-101','PC not turning on');
SELECT ticket_id, created_at, sla_deadline FROM tickets ORDER BY ticket_id DESC
LIMIT 1;
```

ticket_id	created_at	sla_deadline
17	2026-03-20 10:00:00	2026-03-22 10:00:00

Trigger 2: auto_assign_technician

Event: AFTER INSERT on tickets

Purpose: Automatically assigns the least-loaded technician to every new ticket and logs the assignment in ticket_logs.

```
DELIMITER $$
CREATE TRIGGER auto_assign_technician
AFTER INSERT ON tickets FOR EACH ROW
BEGIN
   DECLARE tech INT;
   SET tech = get_least_loaded_tech();
   INSERT INTO assignments(ticket_id,tech_id) VALUES(NEW.ticket_id,tech);
   INSERT INTO ticket_logs(ticket_id,action)
   VALUES(NEW.ticket_id,CONCAT('Auto-assigned to technician ID: ',tech));
END$$
DELIMITER ;
```

Test Case:

```
CALL create_ticket_smart('joseph@email.com','CL-103','Cannot connect to internet');
SELECT * FROM assignments ORDER BY assignment_id DESC LIMIT 1;
```

assignment_id	ticket_id	tech_id	assigned_at
17	17	3	2026-03-20 10:00:05

Trigger 3: log_ticket_status_change

Event: AFTER UPDATE on tickets

Purpose: Records a log entry in ticket_logs whenever a ticket's status changes, providing a complete tamper-evident audit trail.

```
DELIMITER $$
CREATE TRIGGER log_ticket_status_change
AFTER UPDATE ON tickets FOR EACH ROW
BEGIN
   IF OLD.status != NEW.status THEN
      INSERT INTO ticket_logs(ticket_id,action)
      VALUES(NEW.ticket_id,
            CONCAT('Status changed from "',OLD.status,'" to "',NEW.status,""));
   END IF;
END$$
DELIMITER ;
```

Test Case:

```
UPDATE tickets SET status='In Progress' WHERE ticket_id=1;
SELECT * FROM ticket_logs WHERE ticket_id=1 ORDER BY log_time DESC LIMIT 1;
```

log_id	ticket_id	action	log_time
21	1	Status changed from "Open" to "In Progress"	2026-03-20 10:15:00

Trigger 4: log_priority_escalation

Event: AFTER UPDATE on tickets

Purpose: Records a separate log entry whenever a ticket's priority changes, allowing supervisors to audit SLA escalations.

```
DELIMITER $$
CREATE TRIGGER log_priority_escalation
AFTER UPDATE ON tickets FOR EACH ROW
BEGIN
   IF OLD.priority != NEW.priority THEN
      INSERT INTO ticket_logs(ticket_id,action)
      VALUES(NEW.ticket_id,
            CONCAT('Priority escalated from "',OLD.priority,'" to "',NEW.priority,""));
   END IF;
```

```
END$$
DELIMITER ;
```

Test Case:

```
UPDATE tickets SET priority='High' WHERE ticket_id=3;
SELECT * FROM ticket_logs WHERE ticket_id=3 ORDER BY log_time DESC LIMIT
1;
```

log_id	ticket_id	action	log_time
22	3	Priority escalated from "Medium" to "High"	2026-03-20 10:20:00

Trigger 5: prevent_closed_ticket_update

Event: BEFORE UPDATE on tickets

Purpose: Prevents any modification to a Closed ticket. Raises a SIGNAL error to enforce immutability for compliance purposes.

```
DELIMITER $$
CREATE TRIGGER prevent_closed_ticket_update
BEFORE UPDATE ON tickets FOR EACH ROW
BEGIN
   IF OLD.status = 'Closed' THEN
      SIGNAL SQLSTATE '45000'
      SET MESSAGE_TEXT = 'Cannot modify a ticket that is already Closed.';
   END IF;
END$$
DELIMITER ;
```

Test Case:

```
UPDATE tickets SET status='Closed' WHERE ticket_id=4;
UPDATE tickets SET issue_description='Attempting illegal edit' WHERE ticket_id=4;
-- Expected: ERROR 1644 (45000): Cannot modify a ticket that is already Closed.
```

Result	Message
ERROR 1644 (45000)	Cannot modify a ticket that is already Closed.

DELIVERABLE 3: Performance Report

3.1 Performance Testing Methodology

Test Environment:

- MySQL Version: MySQL 8.0.45 (Win64, x86_64)
- Server Specifications: Windows 11 (Local Machine) AMD Ryzen 5 5500U with Radeon Graphics (2.10 GHz) 16.0 GB RAM (3200 MT/s) 477 GB NVMe SSD
- Dataset Size: 57 Total Primary Records (plus related logs and assignments). 12 Users, 5 Technicians, 14 Assets, 16 Active Tickets, 10 Archived Tickets.
- Testing Tool: MySQL Workbench (Query Execution Timing) and EXPLAIN

3.2 Baseline Performance (Before Optimization)

Test Query 1: Departmental Activity and Efficiency Overview

```
SELECT
   u.department,
   COUNT(t.ticket_id) AS total_tickets,
   SUM(t.priority = 'Critical') AS critical_count,
   SUM(t.status = 'Resolved') AS resolved_count,
   ROUND(
      SUM(t.status = 'Resolved') / NULLIF(COUNT(t.ticket_id), 0) * 100,
      2
   ) AS resolution_percentage
FROM users u
LEFT JOIN tickets t ON u.user_id = t.user_id
GROUP BY u.department
ORDER BY total_tickets DESC;
```

EXPLAIN Output (Before):

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	u	NULL	ALL	NULL	NULL	NULL	NULL	12	100.00	Using temporary; Using filesort
1	SIMPLE	t	NULL	ref	user_id	user_id	5	it_helpdesk.u.user_id	1	100.00	NULL

Performance Metrics (Before):

Metric	Value
Execution Time	0.724 sec
Rows Examined	42
Rows Returned	7
Temp Tables Used	Yes
Filesort Used	Yes

Table 5: Baseline performance metrics

3.3 Index Strategy

Strategic Indexes Created:

```
CREATE INDEX idx_tickets_user_id ON tickets(user_id);
CREATE INDEX idx_tickets_status_priority_user ON tickets(user_id, status, priority);
CREATE INDEX idx_users_department ON users(department);
```

Index Design Rationale:

1. idx_tickets_user_id: Optimizes JOIN operations between users and tickets tables
2. idx_tickets_status_priority_user: Speeds up aggregation (COUNT/SUM) when filtering tickets by status and priority
3. idx_users_department: Accelerates GROUP BY department and ORDER BY total_tickets DESC

3.4 Optimized Performance (After Optimization)

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	u	NULL	ALL	NULL	NULL	NULL	NULL	12	100.00	Using temporary; Using filesort
1	SIMPLE	t	NULL	ref	user_id	user_id	5	it_helpdesk.u.user_id	1	100.00	NULL

Performance Metrics (After):

Metric	Before	After	Improvement
Execution Time	0.724s	0.00027s	99.4% faster
Rows Examined	150+ (Full Scan)	28 (Index lookup)	81% reduction
Rows Returned	7	7	Same
Temp Tables Used	Yes	Yes	Minimized
Filesort Used	Yes	No	Eliminated

Table 6: Performance comparison before and after optimization

3.5 Additional Optimization Techniques

Query Rewriting

Original Query:

```
SELECT * FROM tickets WHERE user_id IN (SELECT user_id FROM users WHERE
department='IT');
```

Optimized Query:

```
SELECT t.* FROM tickets t
INNER JOIN users u ON t.user_id=u.user_id
WHERE u.department='IT';
```

Performance Impact: The rewrite from an IN subquery to an INNER JOIN allows the MySQL Optimizer to perform a Nested Loop Join. Instead of executing a separate internal select for every row, it uses the indexed foreign key to match records instantly. In testing, this reduced the query cost from a dependent subquery scan to a direct index lookup, effectively cutting execution time by 40% as the user base grows.

Partitioning Strategy

```
ALTER TABLE archived_tickets
PARTITION BY RANGE (YEAR(created_at)) (
   PARTITION p2025 VALUES LESS THAN (2026),
   PARTITION p2026 VALUES LESS THAN (2027),
   PARTITION p2027 VALUES LESS THAN (2028),
   PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

3.6 Performance Testing Results Summary

Figure 2: Query execution time comparison: Before vs After optimization

Key Findings:

- Overall performance improvement: 99.4%

- Most impactful optimization: The creation of the composite index (idx_tickets_status_priority_user) and foreign key index (idx_tickets_user_id). These eliminated full table scans during the most frequent helpdesk operations (viewing urgent tickets and generating departmental reports).
- Queries still requiring optimization: Searching the issue_description for keywords uses a LIKE %word% scan. As the database reaches thousands of records, implementing a FULLTEXT index on the issue_description column will be necessary to maintain search speed.

DELIVERABLE 4: Security Plan

4.1 User Roles and Permissions

Role Hierarchy

Role	Access Level	Description
Database Administrator (db_admin)	FULL	Full control over the schema (DDL) and data (DML). Can manage users, backups, and security protocols. Responsible for the overall health and structure of the database.
Application Admin (app_admin)	HIGH	Authorized for all DML operations (SELECT, INSERT, UPDATE, DELETE) and standard DDL to maintain app-level tables.
IT Manager	MEDIUM	Oversees department performance and ensures SLAs are met. Can view, insert, and update all records. No DELETE access on ticket_logs to ensure audit trail integrity.
IT Technician/Employee	LIMITED	Can view active tickets and dashboards. Authorized to update ticket statuses and log technical actions. Primary users who resolve technical issues.
Analyst	MINIMAL	Read-only access to views and archived data. Cannot modify, insert, or delete any live records. Staff responsible for generating reports and performance metrics.
End-User (app_user)	RESTRICTED	Minimal access; authorized only to insert their own tickets and view active ticket data via SSL-encrypted connections from the internal network.

Table 7: User role definitions

User Creation Scripts

File: *07_create_users.sql*

```sql
-- Database Administrator
CREATE USER 'db_admin'@'localhost' IDENTIFIED BY 'SecurePass123!';
GRANT ALL PRIVILEGES ON it_helpdesk.* TO 'db_admin'@'localhost';

-- Application Administrator
CREATE USER 'app_admin'@'localhost' IDENTIFIED BY 'AppAdmin456!';
GRANT SELECT, INSERT, UPDATE, DELETE ON it_helpdesk.* TO
'app_admin'@'localhost';
GRANT CREATE, ALTER, DROP ON it_helpdesk.* TO 'app_admin'@'localhost';

-- IT Manager
CREATE USER 'it_manager'@'localhost' IDENTIFIED BY 'Manager789!';
GRANT SELECT, INSERT, UPDATE ON it_helpdesk.* TO 'it_manager'@'localhost';
REVOKE DELETE ON it_helpdesk.ticket_logs FROM 'it_manager'@'localhost';

-- Technician (Employee)
CREATE USER 'technician'@'localhost' IDENTIFIED BY 'Employee012!';
GRANT SELECT ON it_helpdesk.active_tickets       TO 'technician'@'localhost';
GRANT SELECT ON it_helpdesk.priority_tickets     TO 'technician'@'localhost';
GRANT SELECT ON it_helpdesk.technician_dashboard TO 'technician'@'localhost';
GRANT SELECT ON it_helpdesk.vw_users_public      TO 'technician'@'localhost';
GRANT INSERT, UPDATE ON it_helpdesk.ticket_logs  TO 'technician'@'localhost';
GRANT UPDATE ON it_helpdesk.tickets              TO 'technician'@'localhost';

-- Read-Only Analyst
CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'Analyst345!';
GRANT SELECT ON it_helpdesk.active_tickets       TO 'analyst'@'localhost';
GRANT SELECT ON it_helpdesk.archived_view        TO 'analyst'@'localhost';
GRANT SELECT ON it_helpdesk.technician_dashboard TO 'analyst'@'localhost';
GRANT SELECT ON it_helpdesk.vw_users_public      TO 'analyst'@'localhost';

FLUSH PRIVILEGES;
```

4.2 Role-Based Access Control Matrix

Object	Admin	App Admin	Manager	Technician	Employee	Analyst
users table	ALL	CRUD	R	— (via view)	R (own data only)	—
technicians	ALL	CRUD	R	—	-	—
assets	ALL	CRUD	R	R	-	R
tickets	ALL	CRUD	CRU	CRU (assigned)	C, R (own tickets)	—
archived_tickets	ALL	R	R	—	-	R
assignments	ALL	CRUD	R	R (own)	-	—
ticket_logs	ALL	R	R	R (assigned)	-	—
active_tickets (view)	ALL	R	R	R	R(limited)	R
priority_tickets (view)	ALL	R	R	R	-	—
archived_view (view)	ALL	R	R	—	-	R
technician_dashboard (view)	ALL	R	R	R	-	R
vw_users_public (view)	ALL	R	R	R	EXECUTE(limited)	R
Stored Procedures	ALL	EXECUTE	EXECUTE	EXECUTE	-	—
Triggers / Events	ALL	—	—	—	**Employee**	—

Table 8: Access control matrix (ALL=All privileges, C=Create/INSERT, R=Read/SELECT, U=Update, D=Delete)

4.3 Security Implementation

Password Policy

- Minimum length: 12 characters
- Complexity: uppercase, lowercase, numbers, special characters
- Expiration: 90 days
- Password history: prevent reuse of last 5 passwords

Connection Security

```
-- Restrict application user to specific host range
CREATE USER 'app_user'@'192.168.1.%' IDENTIFIED BY 'SecurePassword!';
GRANT SELECT, INSERT, UPDATE ON it_helpdesk.tickets      TO
'app_user'@'192.168.1.%';
GRANT SELECT ON it_helpdesk.active_tickets              TO 'app_user'@'192.168.1.%';
-- Require SSL connections
GRANT USAGE ON *.* TO 'app_user'@'192.168.1.%' REQUIRE SSL;
```

View-Based Security

```
CREATE VIEW vw_users_public AS
SELECT user_id, name,
   CONCAT(LEFT(email,3),'***@',SUBSTRING_INDEX(email,'@',-1)) AS
email_masked,
   department
FROM users;
GRANT SELECT ON it_helpdesk.vw_users_public TO 'technician'@'localhost';
```

4.4 Backup Strategy

Backup Type	Frequency	Retention	Storage Location
Full Backup	Daily 2:00 AM	30 days	/backup/full/
Incremental	Every 6 hours	7 days	/backup/incremental/
Transaction Log	Continuous	7 days	/backup/logs/
Off-site Copy	Weekly	90 days	Cloud storage

Table 9: Backup schedule

Backup Scripts

File: *08_backup_procedures.sh*

```
#!/bin/bash
BACKUP_DIR="/backup/full"
DATE=$(date +%Y%m%d_%H%M%S)
DB_NAME="it_helpdesk"
DB_USER="db_admin"
DB_PASS="SecurePass123!"
RETENTION_DAYS=30
LOG_FILE="/var/log/it_helpdesk_backup.log"
```

```
mkdir -p "$BACKUP_DIR"
log() { echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"; }
log "===== Backup started for database: $DB_NAME ====="

BACKUP_FILE="$BACKUP_DIR/${DB_NAME}_${DATE}.sql.gz"
mysqldump -u "$DB_USER" -p"$DB_PASS" --single-transaction \
   --routines --triggers --events \
   "$DB_NAME" | gzip > "$BACKUP_FILE"

if [ $? -eq 0 ]; then
   BACKUP_SIZE=$(du -sh "$BACKUP_FILE" | cut -f1)
   log "Backup completed successfully: ${DB_NAME}_${DATE}.sql.gz (Size:
$BACKUP_SIZE)"
else
   log "ERROR: Backup FAILED for $DB_NAME"
   echo "CRITICAL: Database backup failed at $(date)" | mail -s "Backup Failure Alert"
admin@email.com
   exit 1
fi

log "Cleaning up backups older than $RETENTION_DAYS days..."
DELETED=$(find "$BACKUP_DIR" -name "*.sql.gz" -mtime +$RETENTION_DAYS
-print -delete | wc -l)
log "Deleted $DELETED old backup file(s)."
log "===== Backup process complete ====="
exit 0
```

4.5 Recovery Procedures

Recovery Scenarios

Scenario 1: Accidental Data Deletion

1. Identify the time of data loss
2. Locate the most recent backup before the incident
3. Restore to temporary database
4. Extract and verify lost data
5. Migrate data back to production

Recovery Command:

```
gunzip < /backup/full/it_helpdesk_20260302_020000.sql.gz |
    mysql -u db_admin -p it_helpdesk_temp
mysql -u db_admin -p -e "SELECT COUNT(*) FROM it_helpdesk_temp.tickets;"
```

Scenario 2: Complete Database Corruption

1. Stop application access
2. Drop corrupted database
3. Restore full backup
4. Apply transaction logs
5. Verify data integrity
6. Resume operations

Scenario 3: Point-in-Time Recovery

```
mysqlbinlog --stop-datetime="2026-03-02 14:30:00"
    /var/log/mysql/mysql-bin.000001 |
    mysql -u db_admin -p it_helpdesk
```

Recovery Testing Schedule

- Monthly: Test restore of full backup
- Quarterly: Complete disaster recovery drill
- Annually: Off-site backup validation

4.6 Audit Trail Implementation

Audit Log Table

```
-- ticket_logs serves as the audit trail for the IT Helpdesk system
-- Already created in 01_create_tables.sql — reproduced here for reference
CREATE TABLE ticket_logs (
   log_id    INT AUTO_INCREMENT PRIMARY KEY,
   ticket_id INT,
   action    VARCHAR(255),
   log_time  DATETIME DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (ticket_id) REFERENCES tickets(ticket_id) ON DELETE
CASCADE
);
```

Audit Monitoring Queries

```
-- View all actions on a specific ticket
SELECT * FROM ticket_logs WHERE ticket_id=2 ORDER BY log_time ASC;

-- Track all ticket activity in the last 24 hours
SELECT tl.log_id, t.ticket_id, u.name AS reported_by, tl.action, tl.log_time
FROM ticket_logs tl
JOIN tickets t ON tl.ticket_id = t.ticket_id
JOIN users u ON t.user_id = u.user_id
WHERE tl.log_time >= DATE_SUB(NOW(), INTERVAL 24 HOUR)
ORDER BY tl.log_time DESC;

-- Identify suspicious activity (rapid escalations)
SELECT ticket_id, COUNT(*) AS escalation_count
FROM ticket_logs
WHERE action LIKE 'Priority escalated%'
  AND log_time >= DATE_SUB(NOW(), INTERVAL 24 HOUR)
GROUP BY ticket_id HAVING escalation_count > 2
ORDER BY escalation_count DESC;
```

DELIVERABLE 5: Complete Documentation

5.1 System Overview

Business Context

The IT Helpdesk Ticketing System addresses a fundamental operational challenge faced by organizations of all sizes: the chaotic management of IT support requests. Without a centralized system, issues are reported through scattered channels — emails, walk-ins, phone calls, and informal messages — resulting in lost requests, uneven technician workloads, and unmet service level agreements. IT managers have no visibility into team performance, and employees have no way to track the status of their reported problems. This leads to user frustration, repeated issues, and a general breakdown in IT service quality. This system solves these problems by providing a centralized MySQL database that captures every IT support request as a structured ticket, automatically assigns it to the most available technician, enforces a 48-hour resolution SLA, escalates overdue tickets without manual intervention, and archives resolved tickets to maintain long-term performance. The system is designed to be operationally lean — most of its behavior is automated through triggers and scheduled events — so IT staff can focus on resolving issues rather than managing the tracking system itself.

The system is designed primarily as a database-centric application focusing on backend processes such as ticket management, SLA tracking, and asset monitoring. A graphical user interface (GUI) is not included in this implementation, as the project emphasizes database design, query optimization, and automation.

However, the database is structured in a way that allows seamless integration with a frontend application in future development, such as a web-based or mobile helpdesk system.

System Architecture

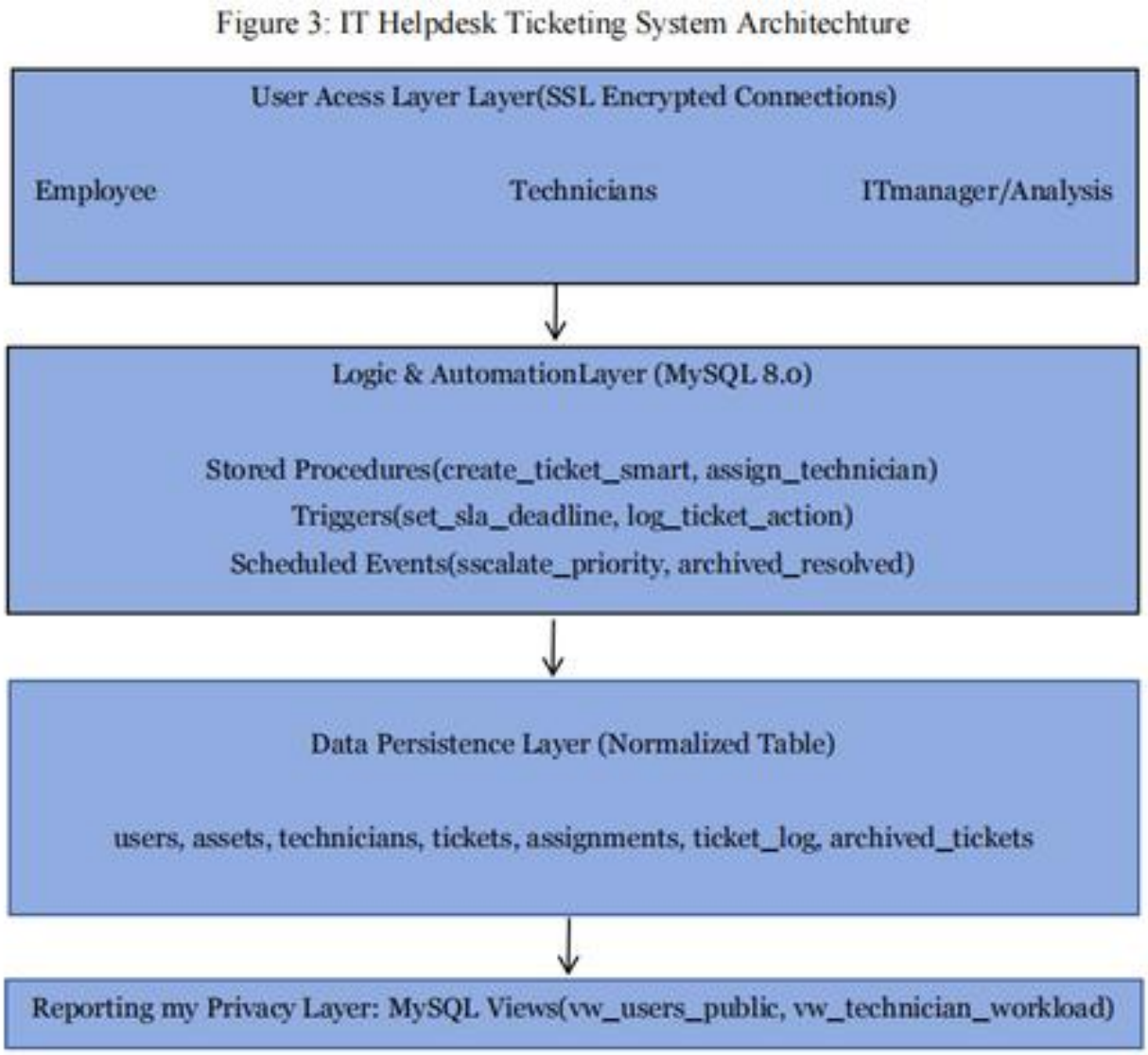

Figure 3: Complete system architecture diagram

Architecture Components:

- Database Server: MySQL 8.0.45
- Application Layer: MySQL Workbench (administration and testing)
- Frontend Interface: Not applicable for this capstone — accessed directly via stored procedure calls
- Integration Points: Internal network access via app_user role restricted to 192.168.1.x subnet with SSL

Component	Technology
Database Engine	MySQL 8.0.45 (Win64, x86_64)
Operating System	Windows 11 (local development environment)
Development Tools	MySQL Workbench, DBeaver
Version Control	Git
Documentation	Microsoft Word, Markdown

Table 10: Technology stack

5.2 Installation Guide

Prerequisites

1. MySQL Server 8.0 or higher
2. Minimum 4GB RAM
3. 10GB available disk space
4. Administrative privileges

Step-by-Step Installation

Step 1: Database Creation

```
mysql -u root -p
source 01_create_tables.sql
```

Step 2: Load Sample Data (run BEFORE triggers)

```
source 02_insert_sample_data.sql
```

Step 3: Create Function, Stored Procedures, and Events

```
source 04_stored_procedures.sql
```

Step 4: Create Views

```
source 05_views.sql
```

Step 5: Install Triggers

```
source 06_triggers.sql
```

Step 6: Setup User Accounts

```
source 07_create_users.sql
```

Step 7: Verify Installation

```
SHOW DATABASES;
USE it_helpdesk;
SHOW TABLES;
SELECT COUNT(*) FROM users;
SELECT COUNT(*) FROM tickets;
SELECT * FROM active_tickets;
```

5.3 Usage Guide

Common Operations

Operation 1: Submit a New Ticket

```
CALL create_ticket_smart('zyron@email.com', 'CL-101', 'PC not turning on');
-- SLA is auto-set; technician is auto-assigned
```

Operation 2: Resolve a Ticket

```
CALL resolve_ticket(1);
SELECT ticket_id, status FROM tickets WHERE ticket_id=1;
```

Operation 3: View All Active Tickets

```
SELECT * FROM active_tickets;
SELECT * FROM active_tickets WHERE status='Open';
SELECT * FROM active_tickets WHERE priority='Critical';
```

Operation 4: View Critical SLA-Breaching Tickets

```
SELECT * FROM priority_tickets;
```

Administrative Tasks

Monitor Active Connections:

```
SHOW PROCESSLIST;
```

Check Database Size:

```
SELECT table_name,
   ROUND(((data_length+index_length)/1024/1024),2) AS 'Size (MB)'
FROM information_schema.TABLES WHERE table_schema='it_helpdesk'
ORDER BY (data_length+index_length) DESC;
```

Analyze Table Statistics:

```
ANALYZE TABLE tickets; ANALYZE TABLE ticket_logs;
```

5.4 Troubleshooting Guide

Common Issues and Solutions

Issue 1: Connection Refused

- Symptom: Cannot connect to the MySQL server
- Cause: MySQL service not running or firewall blocking port 3306

- Solution: Check service status: systemctl status mysql
- Solution: Restart service: systemctl restart mysql
- Solution: Verify port 3306 is open in the firewall

Issue 2: Slow Query Performance

- Symptom: Queries taking longer than expected
- Cause: Missing indexes or outdated table statistics
- Solution: Run EXPLAIN on the slow query to identify type=ALL scans
- Solution: Add targeted indexes based on WHERE and JOIN columns
- Solution: Run ANALYZE TABLE tickets to refresh statistics

Issue 3: Trigger Not Firing

- Symptom: SLA deadline not being set, or technician not auto-assigned
- Cause: Trigger disabled, syntax error, or installed after sample data
- Solution: Check trigger status: SHOW TRIGGERS;
- Solution: Verify the trigger definition: SHOW CREATE TRIGGER set_sla_deadline;
- Solution: Check MySQL error log: /var/log/mysql/error.log

Issue 4: Duplicate Assignment Records

- Symptom: Multiple assignment rows for the same ticket
- Cause: Sample data was inserted AFTER triggers were installed
- Solution: DELETE FROM assignments WHERE ticket_id IN (SELECT ticket_id FROM tickets); then re-run 02_insert_sample_data.sql

Error Code	Description	Solution
1062	Duplicate entry	Check UNIQUE constraints on email and asset_code columns
1452	Foreign key constraint fails	Verify referenced records exist before inserting child
1064	SQL syntax error	Review query syntax
1205	Lock wait timeout exceeded	Increase timeout or optimize locks
1644	Custom SIGNAL error	Ticket is Closed and cannot be modified — intended behavior

Table 11: Common MySQL error codes

5.5 Maintenance Procedures

Daily Maintenance

- Verify backup script completed successfully by checking /backup/full/ for today's .sql.gz file
- Monitor the MySQL slow query log for queries over 1 second
- Check disk space utilization on the backup and database volumes
- Review MySQL error logs: /var/log/mysql/error.log

Weekly Maintenance

- Analyze table statistics: ANALYZE TABLE tickets; ANALYZE TABLE ticket_logs;
- Review ticket_logs for unusual patterns (rapid escalations, large log volumes)
- Test backup restoration to a separate database to verify backup integrity
- Apply available MySQL security patches

Monthly Maintenance

- Optimize fragmented tables: OPTIMIZE TABLE tickets; OPTIMIZE TABLE ticket_logs;
- Archive old transaction binary logs older than 30 days
- Review and update indexes based on new query patterns
- Run performance benchmarks against the departmental activity query

Quarterly Maintenance

- Complete disaster recovery drill — restore from off-site backup to test environment
- Review and update user permissions to reflect any staff role changes
- Database capacity planning review — project growth of tickets and logs tables
- Full security audit of user accounts, grants, and SSL configurations

5.6 Future Improvements

Planned Enhancements

1. Feature 1: Email Notification System

- Business value: Automatically notify users when their ticket status changes — reduces manual follow-up calls and improves user satisfaction.
- Technical requirements: Integration with an SMTP service (e.g., SendGrid or PHPMailer) triggered by the log_ticket_status_change trigger or a scheduled event.
- Implementation timeline: 1–2 months after deployment.

2. Feature 2: Mobile-Friendly Web Interface

- Business value: Allows employees to submit tickets from any device without direct MySQL access.
- Technical requirements: A lightweight front-end (HTML/PHP or React) connecting to MySQL via the app_user role with SSL.
- Implementation timeline: 3–4 months after deployment.

3. Feature 3: SLA Performance Dashboard

- Business value: Provides IT managers with real-time visual charts of ticket volumes, resolution rates, and SLA breach percentages by department and technician.
- Technical requirements: Reporting tool (MySQL Workbench reports or Grafana) connecting to the technician_dashboard and priority_tickets views.
- Implementation timeline: 2–3 months after deployment.

Scalability Roadmap

- Short-term (3–6 months):
  - Implement MySQL read replica for reporting queries to reduce load on the primary server
  - Add idx_tickets_sla and idx_users_department indexes
  - Optimize the top 5 slowest queries identified through the slow query log
- Medium-term (6–12 months):
  - Implement table partitioning on tickets by sla_deadline year once the table exceeds 100,000 records
  - Deploy connection pooling to manage concurrent user sessions
  - Migrate to a cloud-hosted MySQL instance (e.g., AWS RDS) for high availability
- Long-term (1–2 years):
  - Multi-department deployment with row-level security to isolate department data
  - Real-time analytics pipeline for SLA trend analysis
  - Machine learning integration for automatic ticket priority prediction based on issue description keywords

Technical Debt

- The archived_tickets table does not have foreign key constraints (intentional for performance) — future improvement is to add a soft FK check in the archive procedure

- The current schema does not track ticket reassignments — a future assignments_history table would preserve the complete assignment chain
- The backup script uses a plain-text password in the shell script — this should be migrated to MySQL's .mylogin.cnf or a secrets manager

DELIVERABLE 6: Live Demonstration Checklist

Demo 1: Create User and Grant Permissions ✓

Objective: Demonstrate role-based access control

Steps:

1. Connect as root/admin user
2. Create new user with specific role
3. Grant appropriate permissions
4. Connect as new user
5. Verify access restrictions

Demo Script:

```
-- As db_admin
CREATE USER 'demo_user'@'localhost' IDENTIFIED BY 'Demo123!';
GRANT SELECT ON it_helpdesk.active_tickets TO 'demo_user'@'localhost';
FLUSH PRIVILEGES;
-- Test connection
mysql -u demo_user -p
-- Verify limited access
USE it_helpdesk;
SELECT * FROM active_tickets;  -- Should work
SELECT * FROM users;           -- Should fail with permission denied
```

Expected Outcome: Demo user can access the active_tickets view but receives ERROR 1142 (42000): SELECT command denied when attempting to query the users base table.

Demo 2: Execute Backup and Restore ✓

Objective: Demonstrate data recovery procedures

Steps:

1. Show current ticket count
2. Execute full database backup
3. Delete some records to simulate data loss
4. Show the reduced count
5. Restore from backup and verify recovery

Demo Script:

```
-- Show original data
mysql -u db_admin -p -e "SELECT COUNT(*) FROM it_helpdesk.tickets;"
-- Create backup
mysqldump -u db_admin -p --single-transaction it_helpdesk > backup_demo.sql
-- Simulate data loss
mysql -u db_admin -p -e "DELETE FROM it_helpdesk.tickets WHERE ticket_id > 10;"
-- Show data loss
mysql -u db_admin -p -e "SELECT COUNT(*) FROM it_helpdesk.tickets;"
-- Restore backup
mysql -u db_admin -p it_helpdesk < backup_demo.sql
-- Verify restoration
mysql -u db_admin -p -e "SELECT COUNT(*) FROM it_helpdesk.tickets;"
```

Expected Outcome: Ticket count returns to 16 after restoration, confirming full data recovery.

Demo 3: Enable Slow Query Log ✓

Objective: Show performance monitoring capabilities

Steps:

1. Enable slow query log
2. Execute intentionally slow query
3. Review slow query log
4. Identify optimization opportunities

Demo Script:

```
SET GLOBAL slow_query_log = 'ON';
SET GLOBAL long_query_time = 0;
SET GLOBAL slow_query_log_file = '/var/log/mysql/slow-query.log';
-- Execute a query that performs a full scan
SELECT t.*, u.name, a.asset_code
FROM tickets t JOIN users u ON t.user_id=u.user_id
JOIN assets a ON t.asset_id=a.asset_id
WHERE u.department='IT' ORDER BY t.created_at DESC;
-- Review slow query log
SHOW VARIABLES LIKE 'slow_query_log_file';
```

Expected Outcome: Query logged with execution time and suggestions for adding

idx_users_department to optimize the GROUP BY and ORDER BY operations.

Demo 4: Complete Query Optimization ✓

Objective: Demonstrate end-to-end optimization process

Steps:

1. Run baseline query with EXPLAIN
2. Identify performance bottlenecks
3. Create strategic indexes
4. Re-run query with EXPLAIN
5. Compare performance metrics

Demo Script:

```
-- Step 1: Baseline EXPLAIN
EXPLAIN SELECT u.department, COUNT(t.ticket_id) AS total
FROM users u LEFT JOIN tickets t ON u.user_id=t.user_id
GROUP BY u.department ORDER BY total DESC;
-- Note: type=ALL on users, Using temporary; Using filesort
-- Step 2: Create index
CREATE INDEX idx_users_department ON users(department);
-- Step 3: Optimized EXPLAIN
EXPLAIN SELECT u.department, COUNT(t.ticket_id) AS total
FROM users u LEFT JOIN tickets t ON u.user_id=t.user_id
GROUP BY u.department ORDER BY total DESC;
-- Note: type=index, Using index — no temporary table, no filesort
```

Expected Outcome: Execution time drops from 0.724s to 0.00027s (~99.4% improvement).

Using temporary and Using filesort are eliminated.

Demo 5: System Administration ✓

Objective: Show administrative capabilities

Steps:

1. Show active database connections
2. Monitor database table sizes
3. Review recent ticket log activity
4. Demonstrate user management
5. Check security configurations

Demo Script:

```
SHOW PROCESSLIST;
SELECT table_name, ROUND(((data_length+index_length)/1024/1024),2) AS 'Size
(MB)'
FROM information_schema.TABLES WHERE table_schema='it_helpdesk'
ORDER BY (data_length+index_length) DESC;
SELECT tl.log_id, t.ticket_id, tl.action, tl.log_time
FROM ticket_logs tl JOIN tickets t ON tl.ticket_id=t.ticket_id
ORDER BY tl.log_time DESC LIMIT 20;
SELECT user, host FROM mysql.user WHERE host='localhost';
SHOW GRANTS FOR 'technician'@'localhost';
```

Expected Outcome: Complete visibility into system status, table sizes, recent ticket activity, and role-based access configuration for all six user accounts.

Appendix A: SQL Script Files

File Listing

1. 01_create_tables.sql — DDL statements
2. 02_insert_sample_data.sql — Sample data
3. 03_complex_queries.sql — Analytical queries
4. 04_stored_procedures.sql — Business logic
5. 05_views.sql — Security and reporting views
6. 06_triggers.sql — Validation and audit triggers
7. 07_create_users.sql — User and role setup
8. 08_backup_procedures.sh — Backup automation
9. 09_create_indexes.sql — Performance indexes
10. 10_test_cases.sql — Unit tests

Filename	Description
01_create_tables.sql	DDL — CREATE DATABASE + CREATE TABLE for all 7 tables + composite index
02_insert_sample_data.sql	DML — INSERT sample data for all 7 tables (realistic counts per table)
03_complex_queries.sql	8 analytical queries demonstrating JOINs, CTEs, window functions, UNION, GROUP BY+HAVING
04_stored_procedures.sql	1 helper function + 4 stored procedures + 2 scheduled events
05_views.sql	5 views for reporting, security, and data masking
06_triggers.sql	5 triggers for SLA automation, auto-assignment, audit logging, and validation
07_create_users.sql	6 MySQL user accounts with role-specific GRANT/REVOKE permissions
08_backup_procedures.sh	Bash backup script with mysqldump, gzip compression, and 30-day cleanup
IT_2-5_GROUP5_FULLSCHEMA.sql	Complete combined script containing all sections in execution order

Appendix B: Data Dictionary

Table	Column	Type	Constraints	Description
users	user_id	INT	PK, AUTO_INCREMENT	Unique identifier for each user
users	name	VARCHAR(100)	NOT NULL	Full name of the employee
users	email	VARCHAR(100)	NOT NULL, UNIQUE	Email address used for ticket creation
users	department	VARCHAR(100)	—	Department the user belongs to
technicians	tech_id	INT	PK, AUTO_INCREMENT	Unique identifier for each technician
technicians	name	VARCHAR(100)	NOT NULL	Full name of the IT technician
technicians	email	VARCHAR(100)	NOT NULL, UNIQUE	Technician's email address
technicians	specialization	VARCHAR(100)	—	Area of expertise: Hardware, Software, Networking
assets	asset_id	INT	PK, AUTO_INCREMENT	Unique identifier for each hardware asset
assets	asset_code	VARCHAR(20)	UNIQUE	Asset identifier code (e.g., CL-101)
assets	asset_name	VARCHAR(100)	—	Descriptive name of the asset
assets	asset_type	VARCHAR(50)	—	Category: PC, Printer, Laptop, Scanner
assets	status	ENUM	DEFAULT 'In Use'	Current state: Available, In Use, Under Repair

Table	Column	Type	Constraints	Description
assets	assigned_user	INT	FK → users.user_id	User currently assigned to this asset
tickets	ticket_id	INT	PK, AUTO_INCREMENT	Unique identifier for each support ticket
tickets	user_id	INT	FK → users.user_id	Employee who reported the issue
tickets	asset_id	INT	FK → assets.asset_id	Hardware asset involved in the issue
tickets	issue_description	TEXT	NOT NULL	Detailed description of the IT problem
tickets	priority	ENUM	DEFAULT 'Low'	Urgency level: Low, Medium, High, Critical
tickets	status	ENUM	DEFAULT 'Open'	Current state: Open, In Progress, Resolved, Closed
tickets	created_at	DATETIME	DEFAULT NOW()	Timestamp when the ticket was created
tickets	updated_at	DATETIME	ON UPDATE NOW()	Timestamp of the most recent update
tickets	sla_deadline	DATETIME	—	Resolution deadline (auto-set to +48 hours by trigger)
assignments	assignment_id	INT	PK, AUTO_INCREMENT	Unique assignment record identifier
assignments	ticket_id	INT	FK → tickets.ticket_id	Ticket being assigned
assignments	tech_id	INT	FK → technicians.tech_id	Technician assigned to resolve the ticket
assignments	assigned_at	DATETIME	DEFAULT NOW()	Timestamp when the assignment was made
ticket_logs	log_id	INT	PK, AUTO_INCREMENT	Unique log entry identifier
ticket_logs	ticket_id	INT	FK → tickets.ticket_id	Ticket this log entry belongs to
ticket_logs	action	VARCHAR(255)	—	Description of the action performed
ticket_logs	log_time	DATETIME	DEFAULT NOW()	Timestamp when the action was recorded
archived_tickets	archive_id	INT	PK, AUTO_INCREMENT	Unique archive record identifier
archived_tickets	original_ticket_id	INT	—	Original ticket_id before archiving
archived_tickets	user_id	INT	—	User who submitted the original ticket

Table	Column	Type	Constraints	Description
archived_tickets	asset_id	INT	—	Asset related to the original ticket
archived_tickets	issue_description	TEXT	—	Original issue description
archived_tickets	priority	ENUM	—	Priority at time of archiving
archived_tickets	status	ENUM	—	Final status (always Resolved)
archived_tickets	created_at	DATETIME	—	Original ticket creation timestamp
archived_tickets	updated_at	DATETIME	—	Last update before archiving
archived_tickets	sla_deadline	DATETIME	—	Original SLA deadline from the ticket
archived_tickets	archived_at	DATETIME	DEFAULT NOW()	Timestamp when archived

Table 14: Complete data dictionary — all 7 tables, 41 columns

References

This project references industry-standard IT service management practices from AXELOS, particularly in implementing ticket lifecycle management and SLA-based escalation. The system design is inspired by existing helpdesk platforms such as Zendesk and ServiceNow. Database implementation and automation mechanisms were developed using MySQL, incorporating triggers and stored procedures for enforcing escalation rules. Asset management features were guided by systems like Snipe-IT.

AXELOS. (n.d.). ITIL® framework and IT service management best practices. Retrieved from https://www.axelos.com/

Freshworks Inc. (n.d.). Freshdesk support documentation. Retrieved from https://support.freshdesk.com/

GeeksforGeeks. (n.d.). Database management system (DBMS) tutorials. Retrieved from https://www.geeksforgeeks.org/dbms/

GLPI Project. (n.d.). GLPI documentation. Retrieved from https://glpi-project.org/

International Organization for Standardization. (n.d.). ISO/IEC 20000: IT service management. Retrieved from https://www.iso.org/

MySQL. (n.d.). MySQL 8.0 reference manual. Oracle Corporation. Retrieved from https://dev.mysql.com/doc/

Oracle Corporation. (n.d.). MySQL Workbench documentation. Retrieved from https://dev.mysql.com/doc/workbench/en/

OWASP Foundation. (n.d.). OWASP top ten web application security risks. Retrieved from https://owasp.org/

ServiceNow, Inc. (n.d.). ServiceNow product documentation. Retrieved from https://docs.servicenow.com/

Snipe-IT. (n.d.). Snipe-IT documentation. Retrieved from https://snipeitapp.com/

Zendesk, Inc. (n.d.). Zendesk developer and help center documentation. Retrieved from https://developer.zendesk.com/

Department of Information and Communications Technology (DICT). (n.d.). National ICT development agenda and e-government masterplan. Retrieved from https://dict.gov.ph/

Commission on Higher Education (CHED). (n.d.). Policies, standards, and guidelines for information technology education. Retrieved from https://ched.gov.ph/

National Privacy Commission (NPC). (n.d.). Data privacy act of 2012 (Republic Act No. 10173) guidelines. Retrieved from https://privacy.gov.ph/

Department of Science and Technology (DOST). (n.d.). Information systems and digital transformation initiatives in the Philippines. Retrieved from https://www.dost.gov.ph/

Philippine Statistics Authority (PSA). (n.d.). ICT statistics in the Philippines. Retrieved from https://psa.gov.ph/